SIMATS ENGINEERING

CO1-ASSESSMENT TOOL 1-Constrain Based Problem

COURSE NAME: R PROGRAMMING **COURSE CODE: ITA0406**

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.

(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

1. Student Mark Analysis using Vectors

Create an R program to store the marks of 10 students in a vector. Perform the following operations:

a) Display the marks

b) Calculate total and average

c) Find maximum and minimum marks

d) Display the marks greater than 50 using logical-vector operations

e) Display the position of the highest mark.

Code:

```
marks <- c(45, 67, 89, 52, 76, 34, 91, 58, 63, 49)
print(marks)
total <- sum(marks)
average <- mean(marks)
```

print(total)

print(average)

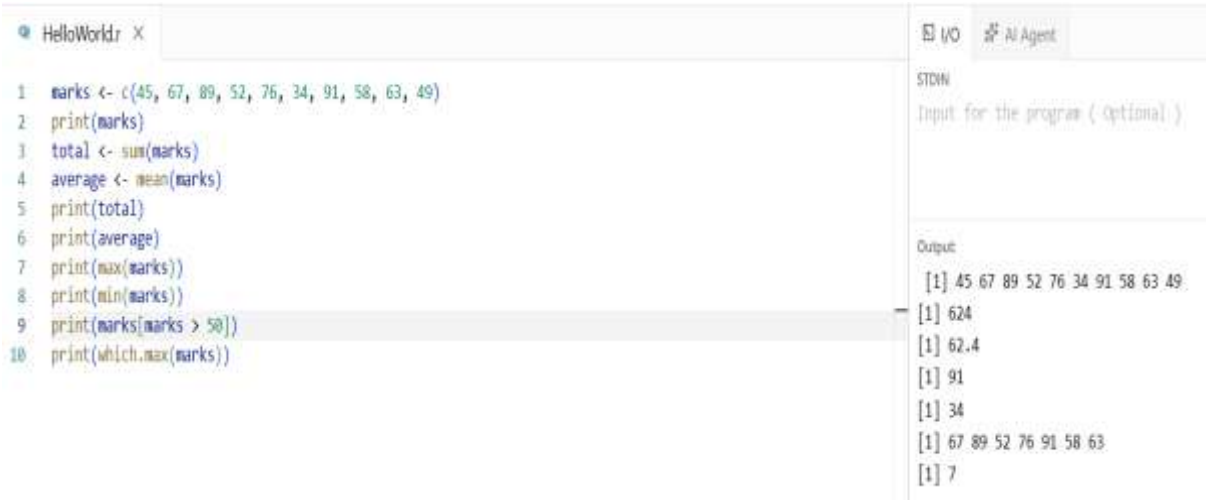
print(max(marks))

print(min(marks))

print(marks[marks > 50])

print(which.max(marks))

Output



2. Student Information using Different Data Types

Develop an R program to store student information such as student name, register number, age, gender, marks, and pass/fail status using appropriate R data types. Display each variable, its value, and its data type. Explain the suitability of each data type.

Code:

name <- "Dhanushree"

register_no <- 101

age <- 20

gender <- "Female"

marks <- 85.5

pass <- TRUE

```
print(name)

print(class(name))

print(register_no)

print(class(register_no))

print(age)

print(class(age))

print(gender)

print(class(gender))

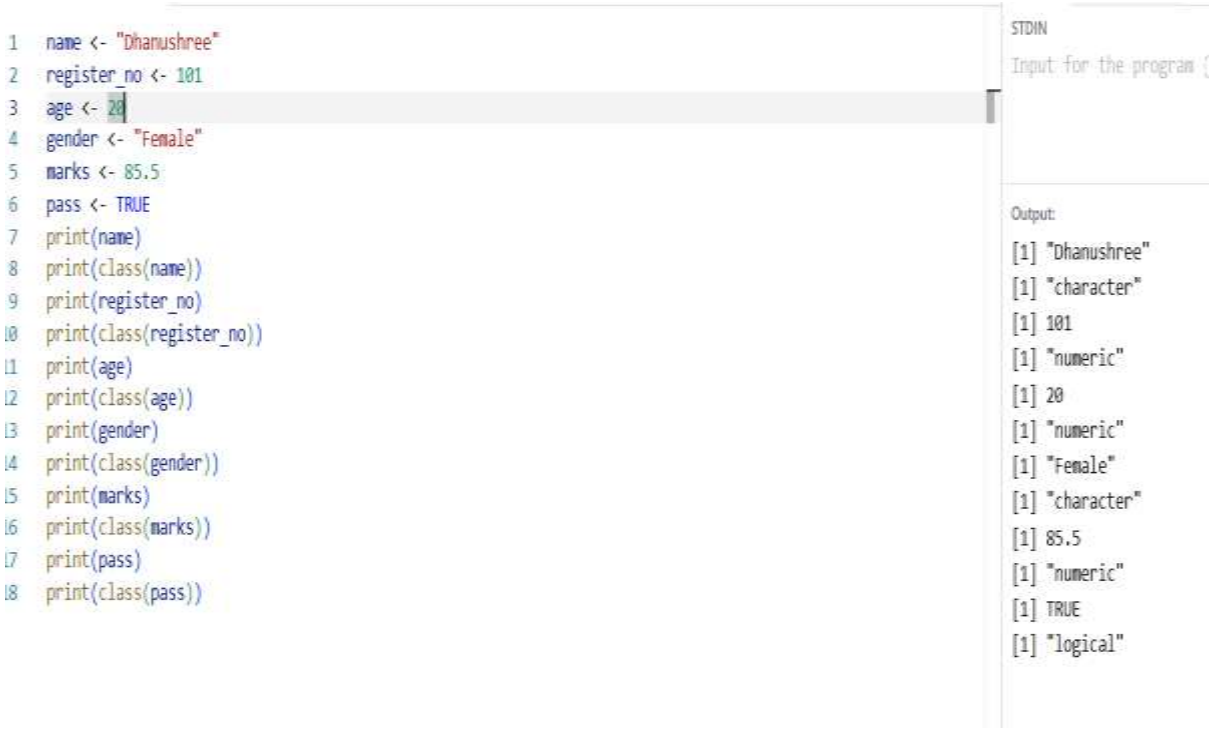
print(marks)

print(class(marks))

print(pass)

print(class(pass))
```

Output:



3. Matrix Operations in R

Create two matrices representing marks obtained by students in two different subjects. Write an R program to perform matrix addition, subtraction, multiplication, transpose, and extraction of selected elements. Explain the operations with suitable output.

Code:

```
A <- matrix(c(10,20,30,40), nrow=2)

B <- matrix(c(5,10,15,20), nrow=2)

print(A)

print(B)

print(A + B)

print(A - B)

print(A %*% B)

print(t(A))

print(A[1,2])
```

Output:



4. Employee Salary Analysis using Lists

Create an R list containing an employee's name, employee ID, department, basic salary, allowances, and deductions. Write an R program to calculate the gross salary and net salary and display the individual components of the list.

Code:

```
employee <- list(
  name = "Ravi",
  id = 101,
  department = "IT",
  basic_salary = 30000,
  allowances = 5000,
  deductions = 2000
)
gross_salary <- employee$basic_salary + employee$allowances
net_salary <- gross_salary - employee$deductions
print(employee)
print(gross_salary)
print(net_salary)
```

Output:



5. Product Sales using Arrays

Create a three-dimensional array to store the sales of three products for two branches over two months. Write an R program to create the array, assign suitable dimension names, access individual values, and calculate the total sales.

Code:

```
sales <- array(
  c(100,120,150,130,110,140,160,170,125,135,155,165),
  dim = c(3,2,2)
)
dimnames(sales) <- list(
  Product = c("P1","P2","P3"),
  Branch = c("B1","B2"),
  Month = c("January","February")
)
print(sales)
print(sales["P1","B1","January"])
print(sum(sales))
```

Output



```
sales <- array(
  c(100,120,150,130,110,140,160,170,125,135,155,165),
  dim = c(3,2,2)
)
dimnames(sales) <- list(
  Product = c("P1","P2","P3"),
  Branch = c("B1","B2"),
  Month = c("January","February")
)
print(sales)
print(sales["P1","B1","January"])
print(sum(sales))
```

```
STDIN
Input for the program (

Output
, , Month = January

        Branch
Product  B1  B2
     P1 100 130
     P2 120 110
     P3 150 140

, , Month = February

        Branch
Product  B1  B2
     P1 160 135
     P2 170 155
     P3 125 165

[1] 100
[1] 1660
```

6. Student Grade Analysis using Factors

Create a factor containing the grades obtained by 10 students (A, B, C, D). Write an R program to display the factor levels, count the number of students in each grade, and access individual factor elements. Explain why a factor is suitable for representing categorical data.

Code:

```
grades <- factor(c("A","B","A","C","B","D","A","C","B","A"))

print(grades)

print(levels(grades))

print(table(grades))

print(grades[3])
```

Output:



```
grades <- factor(c("A","B","A","C","B","D","A","C","B","A"))
print(grades)
print(levels(grades))
print(table(grades))
print(grades[3])
```

```
STDIN
Input for the program (

Output
 [1] A B A C B D A C B A
Levels: A B C D
[1] "A" "B" "C" "D"
grades
A B C D
4 3 2 1
[1] A
Levels: A B C D
```

7. Student Data Frame Problem

Create a data frame containing Student ID, Name, Age, Department, Gender, and Marks for five students. Write an R program to create the data frame, display its contents, access individual columns and rows, and calculate the average, maximum, and minimum marks.

Code:

```
students <- data.frame(
  ID = c(101,102,103,104,105),
  Name = c("Arun","Bala","Cathy","David","Eva"),
  Age = c(20,21,20,22,21),
  Department = c("IT","CSE","IT","ECE","CSE"),
  Gender = c("M","M","F","M","F"),
  Marks = c(85,72,91,65,78)
)
print(students)
print(students$Name)
print(students[1,])
print(mean(students$Marks))
print(max(students$Marks))
print(min(students$Marks))
```

Output:

```
students <- data.frame(
  ID = c(101,102,103,104,105),
  Name = c("Arun","Bala","Cathy","David","Eva"),
  Age = c(20,21,20,22,21),
  Department = c("IT","CSE","IT","ECE","CSE"),
  Gender = c("M","M","F","M","F"),
  Marks = c(85,72,91,65,78)
)
print(students)
print(students$Name)
print(students[1,])
print(mean(students$Marks))
print(max(students$Marks))
print(min(students$Marks))
```

```
STDIN
Input for the program ( Optional )

Output:
  ID  Name Age Department Gender Marks
1 101  Arun  20         IT      M    85
2 102  Bala  21        CSE      M    72
3 103 Cathy  20         IT      F    91
4 104 David  22        ECE      M    65
5 105   Eva  21        CSE      F    78
[1] "Arun"  "Bala"  "Cathy" "David" "Eva"
  ID Name Age Department Gender Marks
1 101 Arun  20         IT      M    85
[1] 78.2
[1] 91
[1] 65
```

8. Logical Vector-Based Student Selection

Create a vector containing the marks of 15 students. Using logical-vector operations, write an R program to identify students who:

a) scored above 75

b) scored between 50 and 75

c) scored below 50

d) passed the examination.

Display the corresponding marks for each category.

Code:

```
marks <- c(45, 82, 67, 91, 38, 74, 55, 88, 49, 76, 62, 33, 95, 71, 43)

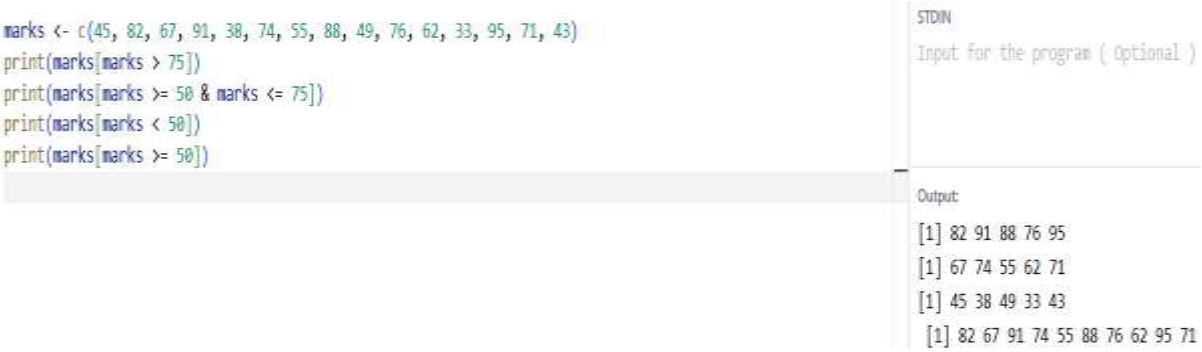
print(marks[marks > 75])

print(marks[marks >= 50 & marks <= 75])

print(marks[marks < 50])

print(marks[marks >= 50])
```

Output:



9. R Data Structure Problem

Develop an R program to represent student information using vectors, lists, matrices, factors, and data frames. Demonstrate how each structure is created and accessed. Explain which type of information is most suitable for each data structure.

Code:

```
marks <- c(80,75,90,65)

print(marks)

student <- list(name="Ravi", age=20, marks=80)

print(student$name)

m <- matrix(1:4, nrow=2)

print(m)

grade <- factor(c("A","B","A","C"))

print(grade)

data <- data.frame(

  Name=c("Ravi","Priya"),

  Marks=c(80,90)

)

print(data)
```

Output:

```
marks <- c(80,75,90,65)
print(marks)
student <- list(name="Ravi", age=20, marks=80)
print(student$name)
m <- matrix(1:4, nrow=2)
print(m)
grade <- factor(c("A","B","A","C"))
print(grade)
data <- data.frame(
  Name=c("Ravi","Priya"),
  Marks=c(80,90)
)
print(data)
```

```
Output:
[1] 80 75 90 65
[1] "Ravi"
     [,1] [,2]
[1,]    1    3
[2,]    2    4
[1] A B A C
Levels: A B C
   Name Marks
1  Ravi    80
2 Priya    90
```

10. R Calculation and Data Structure Problem

Write an R program for a small student result-processing application. Store the marks of five students in vectors, calculate total and average marks, create a result matrix, assign grades using factors, and organize the complete student information in a data frame. Display the final result.

Code:

```
names <- c("Arun","Bala","Cathy","David","Eva")

m1 <- c(80,70,90,60,85)

m2 <- c(75,65,88,55,80)

m3 <- c(85,72,92,65,90)

total <- m1 + m2 + m3

average <- total / 3

grade <- factor(

  ifelse(average >= 80, "A",

  ifelse(average >= 60, "B",

  ifelse(average >= 50, "C", "D")))

)
```

```
result <- cbind(m1, m2, m3, total, average)

students <- data.frame(

  Name = names,

  Subject1 = m1,

  Subject2 = m2,

  Subject3 = m3,

  Total = total,

  Average = average,

  Grade = grade

)

print(students)
```

Output:

```
names <- c("Arun","Bala","Cathy","David","Eva")
m1 <- c(80,70,90,60,85)
m2 <- c(75,65,88,55,80)
m3 <- c(85,72,92,65,90)
total <- m1 + m2 + m3
average <- total / 3
grade <- factor(
  ifelse(average >= 80, "A",
  ifelse(average >= 60, "B",
  ifelse(average >= 50, "C", "D")))
)
result <- cbind(m1, m2, m3, total, average)
students <- data.frame(
  Name = names,
  Subject1 = m1,
  Subject2 = m2,
  Subject3 = m3,
  Total = total,
  Average = average,
  Grade = grade
)
print(students)
```

```
STDIN
Input for the program ( Optional )

Output:
   Name Subject1 Subject2 Subject3 Total Average Grade
1  Arun       80       75       85   240      80     A
2  Bala       70       65       72   207      69     B
3 Cathy       90       88       92   270      90     A
4 David       60       55       65   180      60     B
5   Eva       85       80       90   255      85     A
```

Code of Conduct:

I Dhanushree S/192421218 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student :*Dhanushree S*

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, and constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient approach